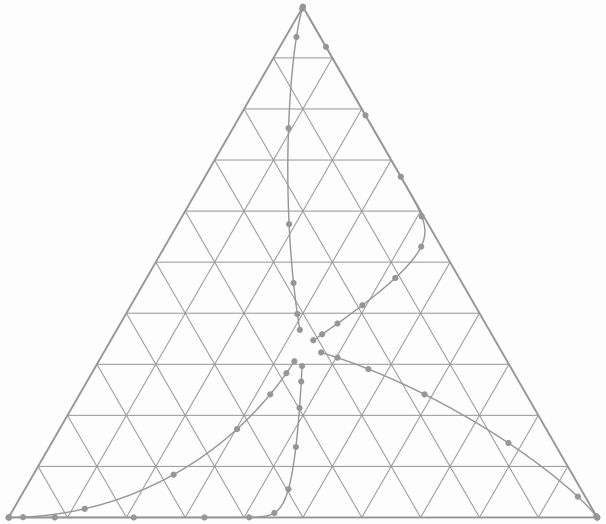
Softmax

and its rounding errors

When float32 rounds the gradient to zero

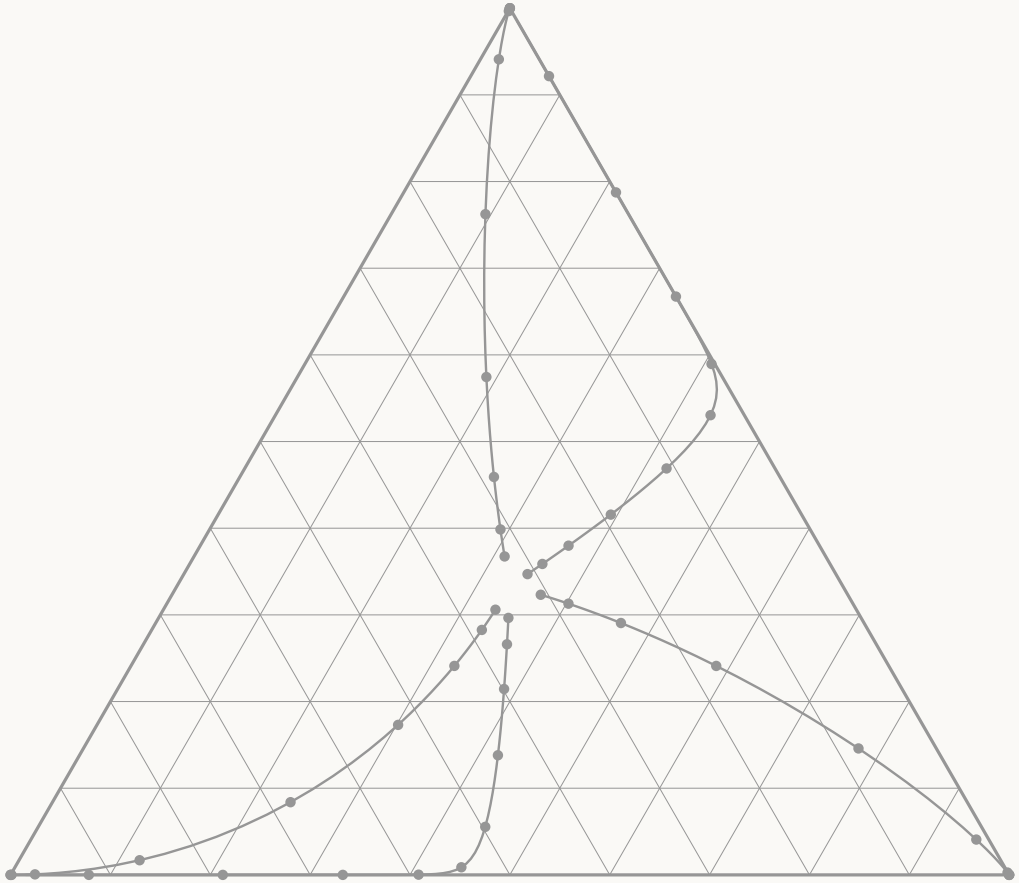


Sadamori Kojaku

Outline

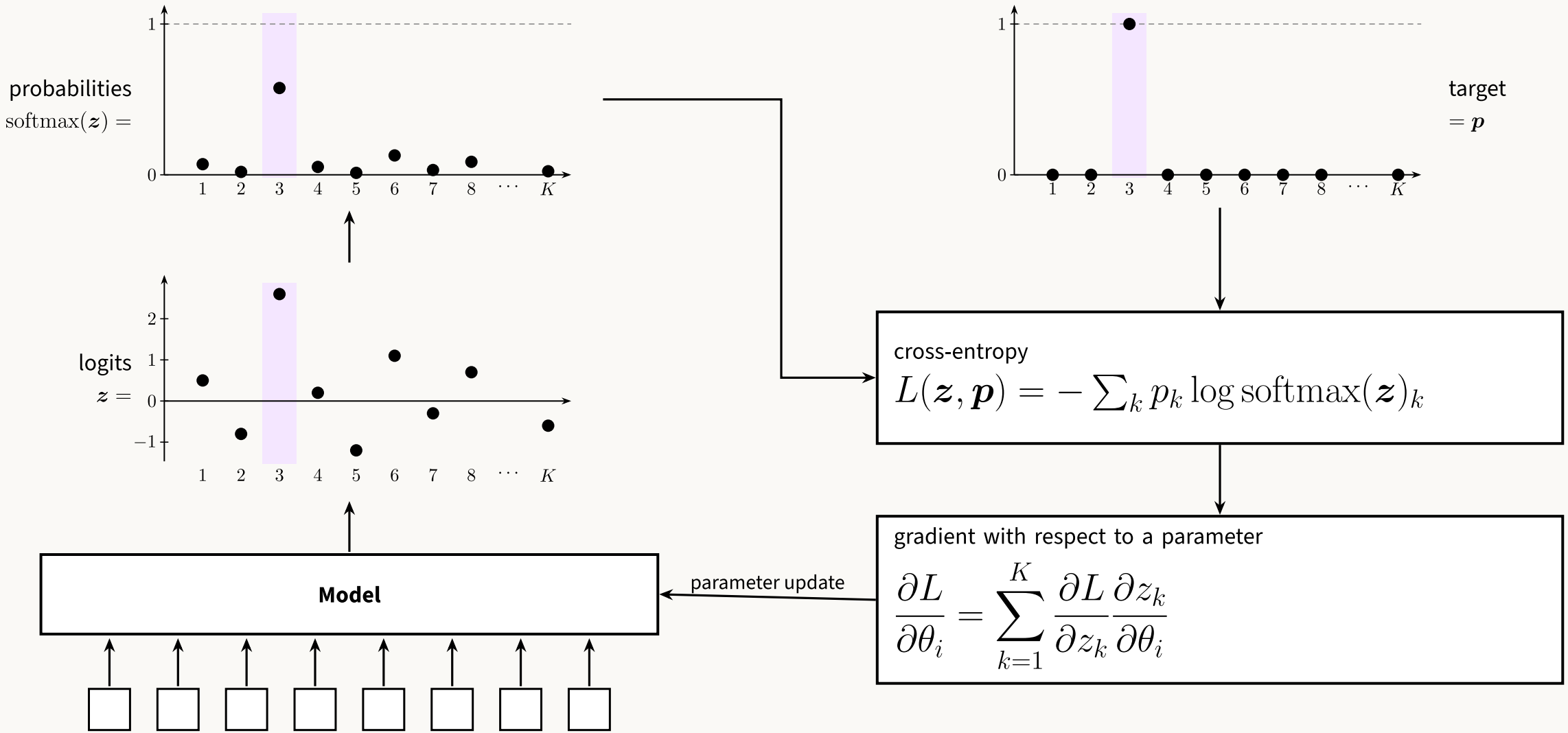
1. Softmax and cross-entropy
2. The logit margin
3. Rounding in float32
4. A formula that keeps the gradient

Softmax and cross-entropy



One training step

The model outputs logits, softmax turns them into probabilities, and the gradient of the cross-entropy updates the parameters.



Softmax and its standard implementation

Softmax turns the logits $\boldsymbol{z} = [z_1, \ldots, z_K]$ into a probability distribution. The standard implementation subtracts the largest logit first, because e^z overflows float32 for $z > 88.7$.

Probabilities

$$\mathrm{softmax}(\boldsymbol{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

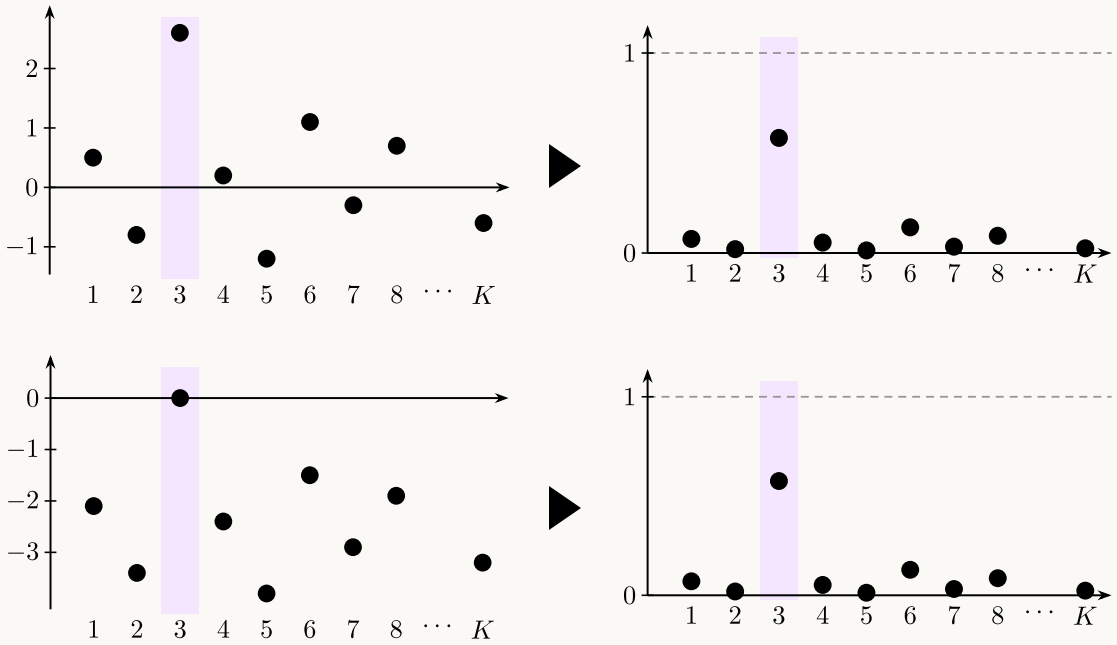
e^z is larger than the largest float32 number (3.4×10^{38}) when $z > 88.7$.

Standard implementation

With $z_{\max} = \max_j z_j$ and $\tilde{z}_k = z_k - z_{\max}$,

$$\mathrm{softmax}(\boldsymbol{z})_k = \frac{e^{\tilde{z}_k}}{\sum_{j=1}^{K} e^{\tilde{z}_j}}$$

The order of the logits is kept, and the largest one becomes 0, so no exponent is positive.



The logit margin

The logit margin m is how far the target logit z_t is above the other logits. The target probability is a function of m alone.

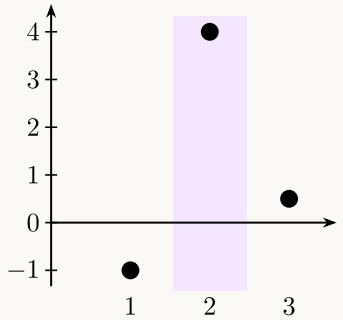
Target logit: z_t

Logit margin

$$m = z_t - \log \sum_{j \neq t} e^{z_j}$$

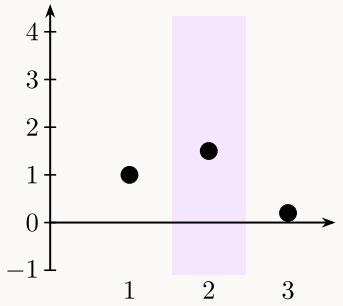
- $\log \sum_j e^{z_j}$ is a smooth version of $\max_j z_j$
- m is how far z_t is above the other logits

A clear winner



$m = 4 - \log(e^{-1} + e^{0.5}) \approx 3.299$

A near tie



$m = 1.5 - \log(e^{1} + e^{0.2}) \approx 0.129$

The target probability in terms of the margin

$$\begin{aligned} e^{-m} &= e^{-z_t + \log \sum_{j \neq t} e^{z_j}} && \text{(definition of } m\text{)} \\ &= e^{-z_t} \sum_{j \neq t} e^{z_j} && (e^{a+b} = e^a e^b,\ e^{\log x} = x) \\ &= \sum_{j \neq t} e^{z_j - z_t} \end{aligned}$$

$$\begin{aligned} \mathrm{softmax}(\boldsymbol{z})_t &= \frac{e^{z_t}}{e^{z_t} + \sum_{j \neq t} e^{z_j}} \\ &= \frac{1}{1 + \sum_{j \neq t} e^{z_j - z_t}} \\ &= \frac{1}{1 + e^{-m}} = \mathrm{sigmoid}(m) \end{aligned}$$

Derivative of the loss

The derivative of the cross-entropy with respect to the logits is $\boldsymbol{q} - \boldsymbol{p}$, where $\boldsymbol{q} = \mathrm{softmax}(\boldsymbol{z})$.

Logits: $\boldsymbol{z} = [z_1, \ldots, z_K]$ Target: $\boldsymbol{p} = [p_1, \ldots, p_K]$

Cross-entropy

$$L(\boldsymbol{z}, \boldsymbol{p}) = -\sum_k p_k \log \mathrm{softmax}(\boldsymbol{z})_k$$

Derivative with respect to a logit

$$\begin{aligned} g_j = \frac{\partial L}{\partial z_j} &= -\sum_{k=1}^{K} p_k \left(\delta_{kj} - q_j\right) \\ &= -\sum_{k=1}^{K} p_k \delta_{kj} + q_j \sum_{k=1}^{K} p_k \\ &= q_j - p_j \end{aligned}$$

For a hard target (1 for the target class t, 0 for the others), $\boldsymbol{p}$ is one-hot:

$$\boldsymbol{p} = [0, \ldots, 0, 1, 0, \ldots, 0]$$

The derivatives are then

$$g_t = q_t - 1 \qquad g_j = q_j \quad (j \neq t)$$

Adam (Kingma & Ba 2015)

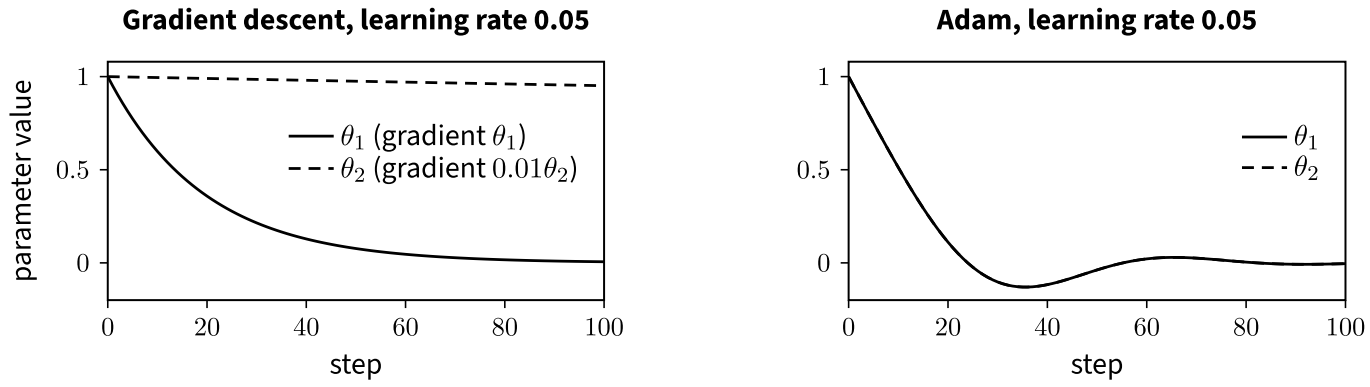
Adam divides each step by a running estimate of the gradient size, so the step hardly depends on how large the gradient is.

Adam update

$$\theta_i \leftarrow \theta_i - \eta \frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \epsilon}$$

- η: learning rate
- $\hat{m}_i$: first moment (recent mean of the gradient)
- $\hat{v}_i$: second moment (recent mean of the squared gradient)
- the step is large when the sign of the gradient is stable

Adam moves both parameters at about the same speed



For the loss

$$L = \frac{1}{2}\left(\theta_1^2 + 0.01\,\theta_2^2\right)$$

- gradient descent: each step scales with the gradient
- Adam: each step is about η, whatever the gradient size

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations.

Outline



1. Softmax and cross-entropy
2. The logit margin
3. **Rounding in float32**
4. A formula that keeps the gradient

Softmax collapse in float32

When the target logit is much larger than the others, the standard cross-entropy rounds the target gradient to 0.

With $\boldsymbol{z} = [9, -9, -9]$ and $\boldsymbol{p} = [1, 0, 0]$:

```
import numpy as np
z = np.array([9.0, -9.0, -9.0], dtype=np.float32)
p = np.array([1.0, 0.0, 0.0], dtype=np.float32)
e = np.exp(z - z.max())
q = e / e.sum()
print(q - p)                # gradient with respect to z
```

```
[0.0000000e+00 1.5229979e-08 1.5229979e-08]
```

The exact value is $[-3.046 \times 10^{-8},\ 1.523 \times 10^{-8},\ 1.523 \times 10^{-8}]$; the target entry is lost.

$$g_t = q_t - p_t = q_t - 1$$

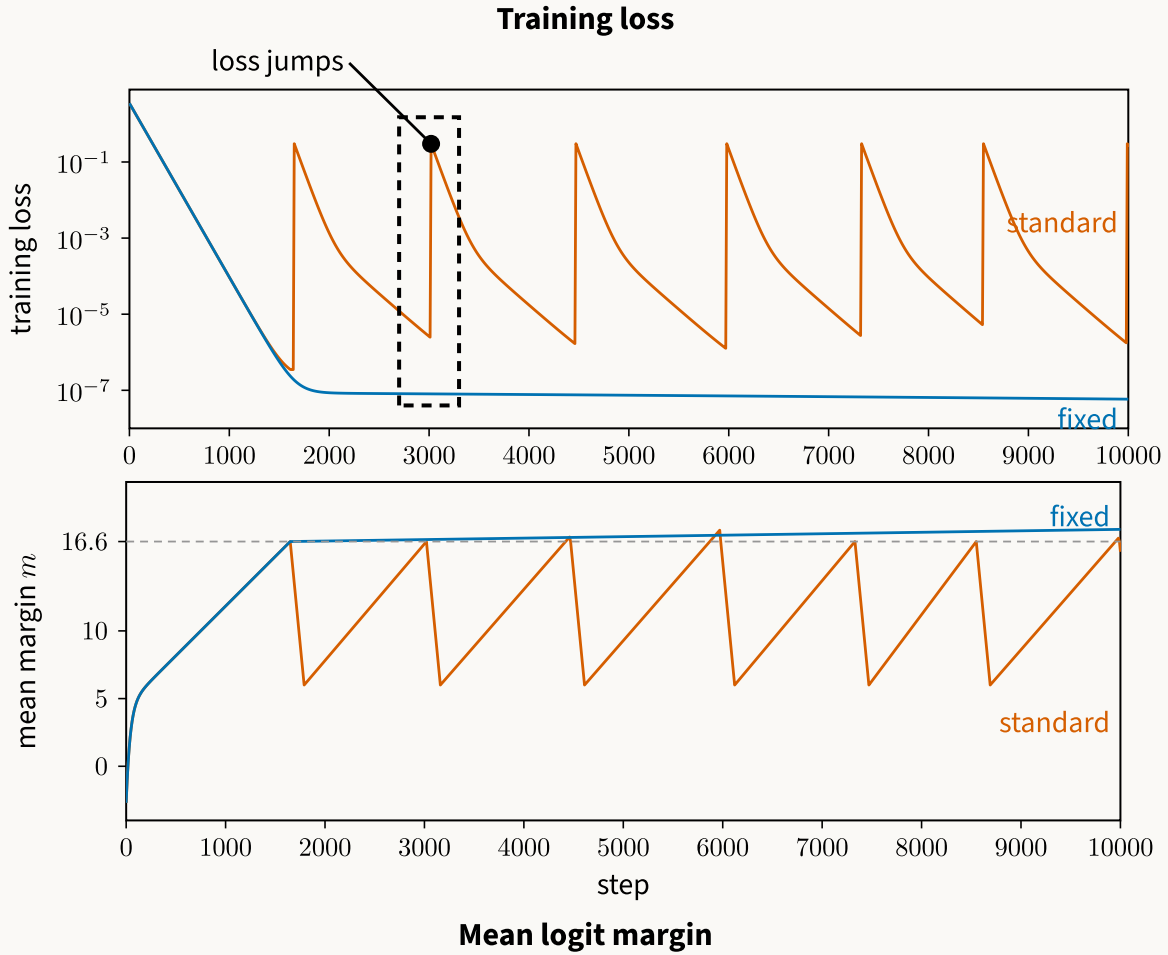
$$= \frac{1}{1 + e^{-m}} - 1$$

Here $m = 18 - \log 2 \approx 17.3$ and $e^{-m} \approx 3.0 \times 10^{-8}$. In float32, $1 + e^{-m}$ rounds to 1 once $e^{-m} < 2^{-24}$, that is, once $m > 24 \log 2 \approx 16.6$.

- it happens when z_t is much larger than every other logit
- z_t stops moving, while the other logits keep moving

Training loss and logit margin

In these synthetic curves, the standard loss jumps each time the mean margin reaches 16.6, and the fixed loss does not.



- standard cross-entropy
  - the loss falls to 10^{-7}, then jumps to 10^{-1}
  - each jump comes when the margin reaches 16.6
  - after the jump, the margin falls to about 6
- fixed cross-entropy
  - the loss stays near 10^{-7}
  - the margin keeps growing past 16.6

The curves are synthetic, made by figures/make_figures.py to show the figure style.

Two formulas for the target gradient

The two formulas are equal in exact arithmetic. The second keeps the small number e^{-m} in the numerator, where float32 has a fine grid.

standard

$$g_t = \frac{1}{1 + e^{-m}} - 1$$

For $m > 16.6$ the fraction rounds to 1, and g_t becomes 0.

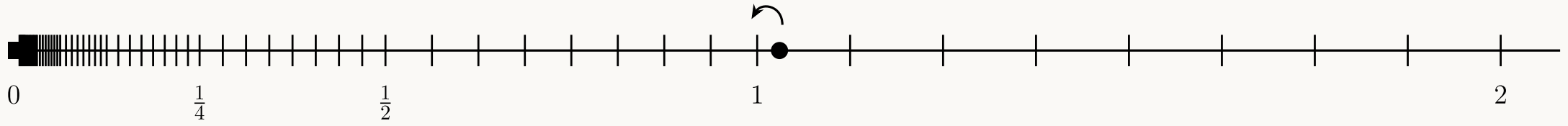
fixed

$$= -\frac{e^{-m}}{1 + e^{-m}}$$

The denominator still rounds to 1, but the numerator keeps its value.

near 0 the grid is finer than near 1

a value near 1 is rounded to 1



Why the loss jumps

After many steps of near-zero gradients, Adam takes a full-size step on the first nonzero gradient.

$$\theta_i \leftarrow \theta_i - \eta \frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \epsilon} - \eta\lambda\theta_i \qquad \cdots \quad \text{parameter update}$$

$$m_i \leftarrow \beta_1 m_i + (1 - \beta_1) \frac{\partial L}{\partial \theta_i} \qquad v_i \leftarrow \beta_2 v_i + (1 - \beta_2) \left(\frac{\partial L}{\partial \theta_i} \right)^2$$

$$\hat{m}_i = \frac{m_i}{1 - \beta_1^\tau} \qquad \hat{v}_i = \frac{v_i}{1 - \beta_2^\tau} \qquad \cdots \quad \text{moment updates}$$

- as the model fits the training data, m grows and q_t approaches 1
- for an example with $m > 16.6$, $g_t = 0$ and every other g_j is a small positive number
- once every example has $m > 16.6$, the gradient is near 0, and $\hat{m}_i$ and $\hat{v}_i$ decay
- when a few examples fall below 16.6 again, $\hat{m}_i / (\sqrt{\hat{v}_i} + \epsilon)$ jumps
- the large step raises the loss

Why the loss jumps

After many steps of near-zero gradients, Adam takes a full-size step on the first nonzero gradient.

$$\theta_i \leftarrow \theta_i - \eta \frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \epsilon} - \eta\lambda\theta_i \quad \cdots \quad \text{parameter update}$$

$$m_i \leftarrow \beta_1 m_i + (1 - \beta_1) \frac{\partial L}{\partial \theta_i}$$

$$\hat{m}_i = \frac{m_i}{1 - \beta_1^\tau}$$

$$v_i \leftarrow \beta_2 v_i + (1 - \beta_2) \left(\frac{\partial L}{\partial \theta_i}\right)^2$$

$$\hat{v}_i = \frac{v_i}{1 - \beta_2^\tau} \quad \cdots \quad \text{moment updates}$$

- as the model fits the training data, m grows and q_t approaches 1
- for an example with $m > 16.6$, $g_t = 0$ and every other g_j is a small positive number
- once every example has $m > 16.6$, the gradient is near 0, and $\hat{m}_i$ and $\hat{v}_i$ decay
- when a few examples fall below 16.6 again, $\hat{m}_i / (\sqrt{\hat{v}_i} + \epsilon)$ jumps
- the large step raises the loss

Why the loss jumps

After many steps of near-zero gradients, Adam takes a full-size step on the first nonzero gradient.

this term jumps

$$\theta_i \leftarrow \theta_i - \eta \frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \epsilon} - \eta\lambda\theta_i \qquad \cdots \text{ parameter update}$$

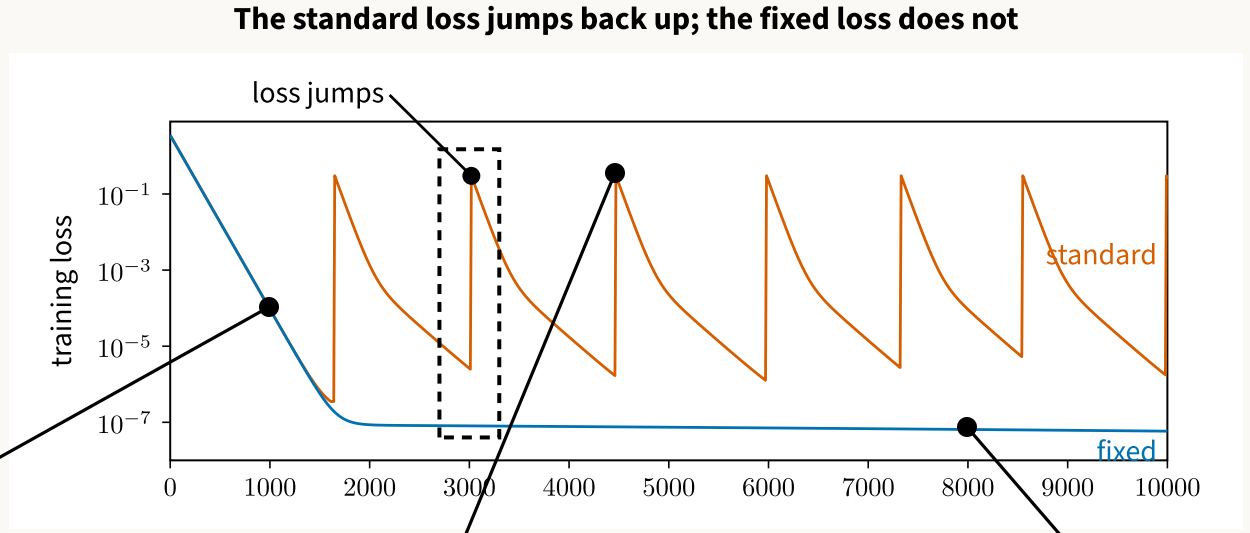
$$m_i \leftarrow \beta_1 m_i + (1 - \beta_1) \frac{\partial L}{\partial \theta_i} \qquad v_i \leftarrow \beta_2 v_i + (1 - \beta_2) \left(\frac{\partial L}{\partial \theta_i} \right)^2$$

$$\hat{m}_i = \frac{m_i}{1 - \beta_1^\tau} \qquad \hat{v}_i = \frac{v_i}{1 - \beta_2^\tau} \qquad \cdots \text{ moment updates}$$

- as the model fits the training data, m grows and q_t approaches 1
- for an example with $m > 16.6$, $g_t = 0$ and every other g_j is a small positive number
- once every example has $m > 16.6$, the gradient is near 0, and $\hat{m}_i$ and $\hat{v}_i$ decay
- when a few examples fall below 16.6 again, $\hat{m}_i / (\sqrt{\hat{v}_i} + \epsilon)$ jumps
- the large step raises the loss

A quoted figure with notes

A figure taken from a paper sits on a white panel, with its source under it and the notes below it.



Synthetic curves. A quoted figure gets its source here, e.g. "Figure 2 of Author et al. (2026)".

Before the first jump
both losses fall at the same rate

A jump
the standard loss rises to about 0.3

The fixed loss
stays near its minimum

A box in sections

A box can hold a header row, a rule across it, and columns separated by grey rules.

Softmax with temperature

$\mathrm{softmax}(\boldsymbol{z}/T)$ is flat for a large T and puts all mass on $\arg\max_k z_k$ as $T \to 0$.

$$q_k = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}}$$

- $T = 1$: the usual softmax
- $T \to \infty$: $q_k \to 1/K$ for every k
- $T \to 0$: $q_k \to 1$ for the largest logit, 0 for the others

Where T appears

distillation · sampling · contrastive loss · annealing

Two names for one thing aside

The inverse temperature $\beta = 1/T$ is used in physics; machine learning uses T.

Notation

The symbols used in this deck.

$\boldsymbol{z} = [z_1, \ldots, z_K]$	logits, one per class
$\boldsymbol{q} = \mathrm{softmax}(\boldsymbol{z})$	predicted probabilities
$\boldsymbol{p}$	target distribution; one-hot for a hard target
m	logit margin of the target class t
$g_j = \partial L / \partial z_j$	derivative of the loss with respect to logit j

Two rounding problems in softmax



The standard formulas for softmax and cross-entropy run into float32 limits at both ends of the range of e^z.

- **Overflow** at large logits
  - e^z is larger than the largest float32 number when $z > 88.7$
  - **subtracting $\max_j z_j$ before the exponential avoids it**
- **Collapse** at a large margin
  - $1 + e^{-m}$ rounds to 1 when $m > 16.6$, so the target gradient g_t becomes 0
  - **writing g_t with e^{-m} in the numerator keeps it**

Summary



- softmax and cross-entropy are computed in float32, which has a fine grid near 0 and a coarse grid near 1
  - $\mathrm{softmax}(\boldsymbol{z})_t = \mathrm{sigmoid}(m)$, with m the logit margin
  - $g_t = q_t - 1$ rounds to 0 once $m > 24 \log 2 \approx 16.6$
- with Adam, a long run of zero target gradients ends in a large step
  - $\hat{m}_i$ and $\hat{v}_i$ decay while the gradient is near 0
  - the first nonzero gradient after that gives a step of about η
- the formula $g_t = -e^{-m}/(1 + e^{-m})$ is equal in exact arithmetic and keeps g_t in float32